SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology

in

Computer Science & Engineering

By

M V Akshaya (VTU25623)

10211CS224

Full Stack Application Development

Slot : E1

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D

INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled **"Smart Campus Event Management System"** by **M V Akshaya (VTU25623)** has been carried out in Winter semester of Academic year 2025–2026 (WS2526).

Signature of Faculty

Dr.Mageshwari .M M.E,Ph.D

Assistant professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Assistant professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(M V Akshaya)
Date: 06/05/2026

ABSTRACT

The **Smart Campus Event Management System** is a full-stack web application designed to simplify and automate the management of campus events in educational institutions. Traditional event management methods involve manual processes leading to inefficiencies, delays, and poor coordination. This project addresses these challenges by providing a centralized digital platform for efficient event creation, management, and participation.

The system incorporates secure **role-based authentication** using Spring Security, allowing administrators to create and manage events while students can browse available events and register online. The application is built using **Spring Boot 3.2.4** for the backend, **Thymeleaf** for dynamic frontend rendering, and an **H2 in-memory database** for data storage. It follows the **Model-View-Controller (MVC)** architectural pattern and exposes **RESTful API endpoints** for potential mobile integrations.

Key features include real-time event availability tracking, OTP-based email verification for student registration, seat capacity management, event categorization by type and department, and a comprehensive admin dashboard with analytics. The system demonstrates effective use of modern Java web technologies to build a scalable, secure, and user-friendly campus event management solution.

Keywords: Spring Boot, MVC Architecture, Campus Events, Role-Based Authentication, Spring Security, Thymeleaf, H2 Database, Spring Data JPA, OTP Verification, RESTful API

LIST OF FIGURES

1.1 System Architecture of the Smart Campus Event Management System 3

3.1 System Flow Diagram of the Smart Campus Event Management System 9

4.1 MVC Architecture of the Smart Campus Event Management System 11

4.2 Data Flow Diagram (DFD Level 0) of the Campus Event Management System . 12

5.1 Home Page — Trending Events, Platform Statistics, and Navigation 16

5.2 Events Listing Page — Search, Filter by Department, Type, and Date 16

5.3 Event Detail Page — Cloud Computing & DevOps Webinar with Registration Panel 17

5.4 OTP Verification Page — 6-Digit Email OTP for Secure Registration 17

5.5 Admin Portal Login — Secure Authentication for Administrators 18

5.6 Admin Dashboard — Analytics, Events by Type, and Top Events by Registrations 18

LIST OF TABLES

5.1 Comparison of Existing Systems and Proposed System 15

7.1 Mapping of Project to Complex Engineering Problem (CEP) 22

7.2 Mapping of Project to Complex Engineering Activities (CEA) 23

7.3 SDG Mapping for the Project 24

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DTO	Data Transfer Object
H2	Hypersonic 2 (In-Memory Database)
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JS	JavaScript
MVC	Model-View-Controller
ORM	Object Relational Mapping
OTP	One-Time Password
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

TABLE OF CONTENTS

ABSTRACT iii

LIST OF FIGURES iv

LIST OF TABLES v

LIST OF ACRONYMS AND ABBREVIATIONS vi

1 INTRODUCTION 1

1.1 Introduction 1

1.2 Aim of the Project 1

1.3 Scope of the Project 2

1.4 Framework 2

2 SURVEY OF SIMILAR APPLICATIONS IN MARKET 4

2.1 Eventbrite 4

2.2 Cvent 5

2.3 Whova and Accelevents 5

2.4 Comparison with Proposed System 5

3 FRAMEWORK DESCRIPTION 7

3.1 Frontend Implementation 7

3.2 Backend Implementation 8

3.3 Database Implementation 9

4 METHODOLOGIES 11

4.1 Project Architecture 11

4.2 Data Flow Diagram 12

4.3 Module 1: User Management 12
4.4 Module 2: Event Management (Core Processing) 13
4.5 Module 3: Registration and Database Management 13

5 RESULTS AND DISCUSSIONS **15**

6 CONCLUSION AND FUTURE ENHANCEMENTS **19**
6.1 Conclusion 19
6.2 Future Enhancements 19

7 ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG **21**
7.1 Mapping of the Project to Complex Engineering Problem (CEP) 21
7.2 Mapping of the Project to Complex Engineering Activities (CEA) 21
7.3 SDG Mapping (CEP Section) 21

References **21**

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid growth of digital education has significantly increased the demand for structured, automated event management systems in academic institutions. Traditional campus event management relies on manual processes — physical notice boards, in-person registrations, and paper-based tracking — which are inefficient, error-prone, and lack real-time coordination.

To overcome these challenges, this project proposes a **Smart Campus Event Management System (SCEMS)**, a full-stack web application that provides a centralized digital platform for campus event discovery, registration, and administration. The system supports two distinct user roles: **students** who can browse events, register online, and receive OTP-based email verification; and **administrators** who can create, update, and delete events and monitor registrations through a dedicated analytics dashboard.

The application is built on **Spring Boot 3.2.4** following the **MVC architecture**, secured by **Spring Security**, with **Thymeleaf** rendering dynamic HTML views and an **H2 in-memory database** for data persistence during development.

1.2 Aim of the Project

The main aim of this project is to design and develop a user-friendly and secure Smart Campus Event Management System that:

- Provides secure role-based authentication distinguishing students and administrators

- Enables administrators to create, update, and delete campus events with seat limits
- Allows students to browse, search, and filter events by department, type, and date
- Implements OTP-based email verification for student event registration
- Delivers real-time seat availability tracking and registration management
- Exposes RESTful API endpoints for potential mobile or third-party integrations
- Provides administrators with an analytics dashboard for event and registration insights

1.3 Scope of the Project

The scope of the Smart Campus Event Management System includes:

- A web-based platform supporting secure role-based access for students and administrators
- Event creation and management supporting 8+ event types: Seminar, Webinar, Workshop, Lecture, Conference, Cultural, Hackathon, and more
- Advanced event search and filtering by title, department, event type, and date range
- Student registration with OTP email verification via the `/register` endpoint
- Admin dashboard with analytics: total events, upcoming events, total registrations, event categories, events by type, and top events by registration count
- RESTful API layer via `EventApiController` for asynchronous frontend operations

The current system operates with an H2 in-memory database suitable for development and testing, with a strong foundation for future cloud deployment and production database migration.

1.4 Framework

The project is developed using the **Spring Framework** with the following key components:

- **Spring Boot 3.2.4** — Provides rapid application setup with embedded Tomcat server

- **Spring Web MVC** — Implements the Model-View-Controller pattern
- **Spring Security** — Role-based authentication; `/admin/**` endpoints are secured
- **Spring Data JPA / Hibernate** — ORM-based database interaction with automatic schema generation[Figure 1.1]
- **Thymeleaf** — Server-side templating engine with Spring Security extras for conditional UI rendering
- **Apache Maven** — Dependency management and build automation
- **H2 Database** — Lightweight in-memory relational database for development
- **Lombok** — Reduces boilerplate in entity and service classes
- **Java 17** — Underlying programming language

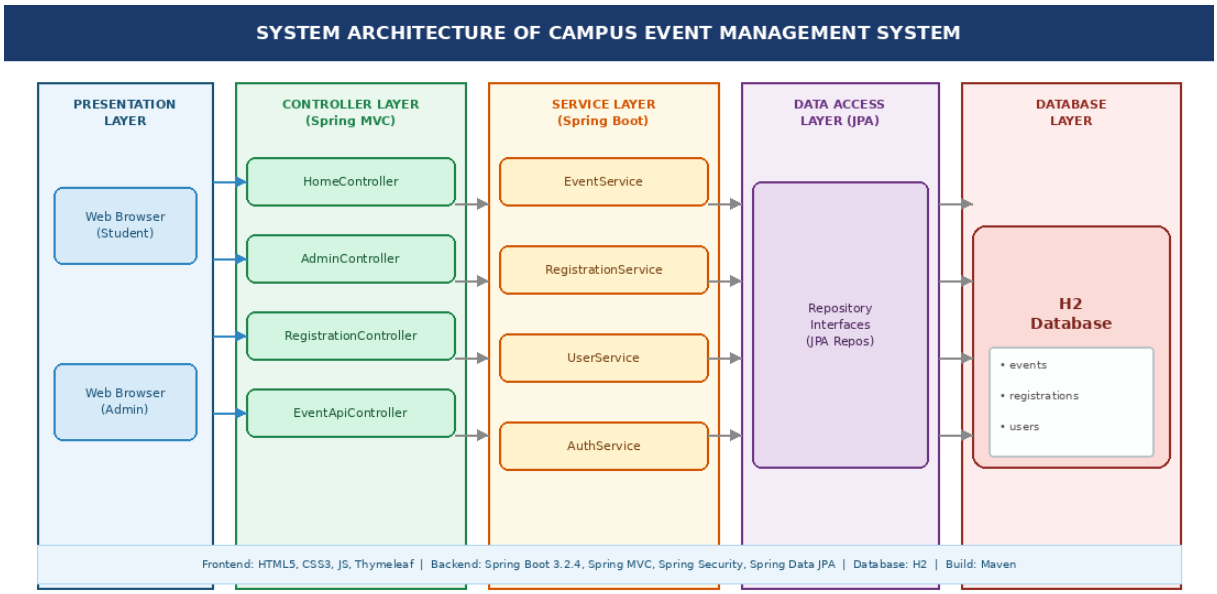

Figure 1.1: System Architecture of the Smart Campus Event Management System

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

The rapid advancement of web technologies has led to the development of various event management platforms. A study of existing systems helps in understanding their features, advantages, and limitations, which assists in designing an improved, campus-specific application[13][14].

2.1 Eventbrite

Eventbrite (https://www.eventbrite.com) is one of the most widely used event management platforms globally. It allows organizers to create events, manage registrations, and issue digital tickets[15].

Advantages:

- Easy-to-use interface for creating and promoting events
- Supports paid and free events with ticket management
- Integration with payment gateways

Limitations:

- Charges service fees for paid events
- Not tailored for academic or campus-specific workflows
- Lacks role-based access for students vs. faculty administrators

2.2 Cvent

Cvent (https://www.cvent.com) is an enterprise-level event management platform used by large organizations and academic institutions for conferences and large events.

Advantages:

- Comprehensive analytics and reporting
- Attendee engagement tools and end-to-end event lifecycle management
- Supports virtual and hybrid events

Limitations:

- Enterprise pricing makes it inaccessible for student-level projects
- Overly complex for simple campus event workflows
- No institution-specific access control

2.3 Whova and Accelevents

Whova (https://whova.com) and Accelevents (https://accelevents.com) provide all-in-one solutions including registration, communication, and analytics with support for virtual events[11][12].

Limitations:

- Primarily designed for professional conferences, not academic environments
- Lack department-wise event categorization and institution-specific access control
- Paid subscription models unsuitable for campus use

2.4 Comparison with Proposed System

The proposed Smart Campus Event Management System overcomes the limitations of existing platforms by providing:

- **Campus-specific workflows** — Department categorization, event types tailored for academic institutions

- **Role-based access** — Strict separation between student and administrator privileges
- **OTP-based verification** — Secure student registration with email OTP confirmation
- **Zero cost** — Free, open-source solution built with standard Java technologies
- **RESTful API** — Extensible architecture for future mobile integrations

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend is developed using **HTML5**, **CSS3**, and **JavaScript** with **Thymeleaf** as the server-side templating engine[1]. Thymeleaf integrates seamlessly with Spring Boot, enabling dynamic data binding from backend controllers to HTML views. Thymeleaf Security extras are used to conditionally render UI elements based on the user's authentication status (e.g., displaying the Admin button only for authenticated administrators). The dark-themed, responsive UI is styled using custom CSS with a modern purple/indigo color scheme. Development is carried out using Eclipse IDE[3][6].

3.1.2 Structure of the Project

The frontend files are organized under `src/main/resources/`:

- `templates/` — Thymeleaf HTML templates: `home.html`, `events.html`, `event-detail.htm` `admin-dashboard.html`, `admin-login.html`, `otp-verify.html`, and others
- `static/css/` — Custom CSS stylesheets for dark theme, card layouts, and responsive grids
- `static/js/` — JavaScript for live filtering, OTP input handling, and dynamic UI
- `static/images/` — Event category icons and UI assets

3.1.3 Execution Procedure

The frontend is served as part of the Spring Boot application. On startup, users access the system through a browser at `http://localhost:8080`. The home page displays trending events and platform statistics. Users navigate to the events page to browse and filter events, then click "Register Now" to initiate OTP-based verification. Admin users access the secured dashboard at `/admin/dashboard[7][9]`.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed in **Java 17** using **Spring Boot 3.2.4**. Key dependencies managed via Apache Maven include:

- **spring-boot-starter-web** — Spring MVC and embedded Tomcat
- **spring-boot-starter-security** — Authentication and authorization
- **spring-boot-starter-data-jpa** — JPA/Hibernate ORM
- **spring-boot-starter-thymeleaf + thymeleaf-extras-springsecurity6** — Templating
- **spring-boot-starter-mail** — OTP email sending
- **lombok** — Boilerplate reduction
- **h2** — In-memory database

3.2.2 Structure of the Project

The backend follows a layered architecture under `src/main/java/com/campus/events/`:

- `config/` — `SecurityConfig.java`: configures Spring Security, role-based access rules
- `controller/` — `HomeController`, `AdminController`, `RegistrationController`, `EventApiController`
- `model/` — JPA Entities: `Event`, `EventType` (enum), `Registration`
- `repository/` — `EventRepository`, `RegistrationRepository` (extend `JpaRepository`

- `service/` — Business logic: `EventService`, `RegistrationService`, `OtpService`, `EmailService`
- `dto/` — Data Transfer Objects for API responses
- `exception/` — Custom exception handlers (`@ControllerAdvice`)

3.2.3 Execution Procedure

The application is started by running the Spring Boot main class or using Maven: `mvn spring-boot:run`. [Figure 3.1] The embedded Tomcat server initializes, Hibernate creates the H2 schema, and the application is accessible at `http://localhost:8080`. All incoming HTTP requests flow through controllers → services → repositories → H2 database, with Thymeleaf views rendered as responses.

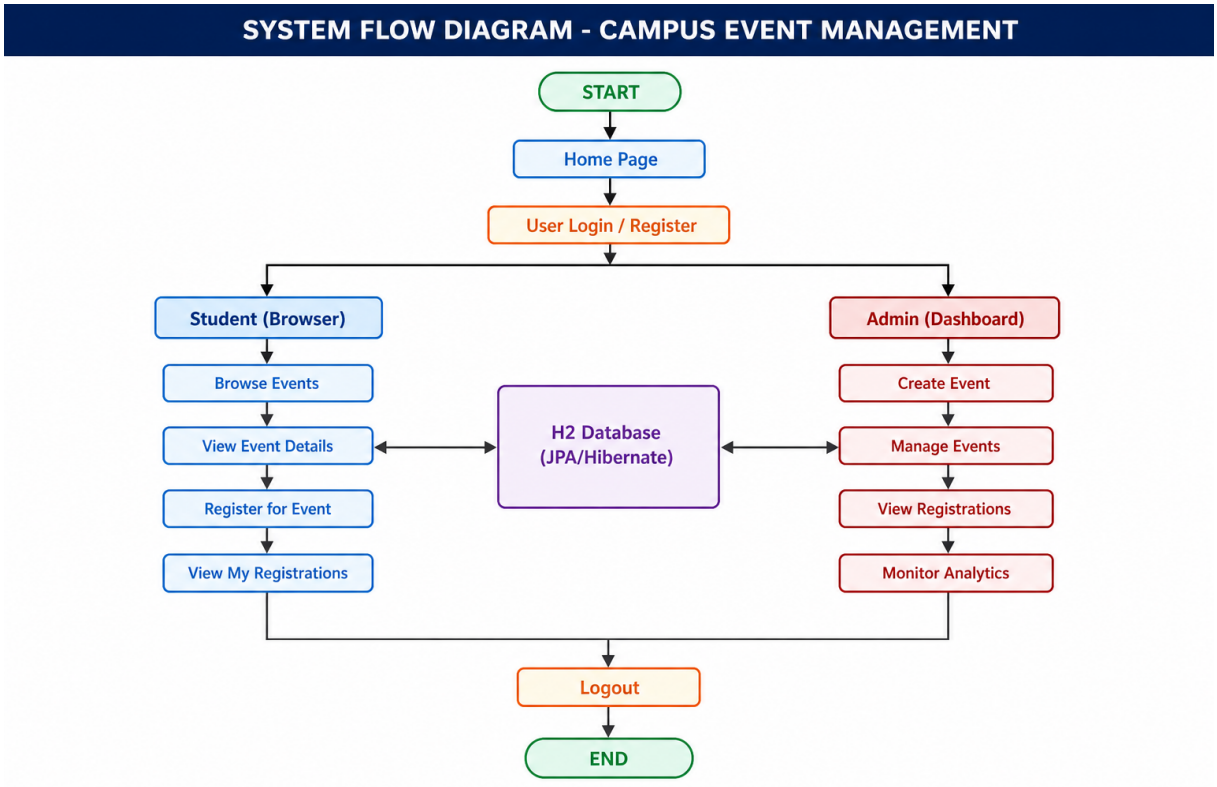

Figure 3.1: System Flow Diagram of the Smart Campus Event Management System

3.3 Database Implementation

3.3.1 Database Configuration

The project uses the **H2 in-memory database**, configured in `application.properties`:

```
spring.datasource.url=jdbc:h2:mem:campusdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Listing 3.1: H2 Database Configuration

The H2 console (accessible at `/h2-console`) allows developers to inspect database tables during runtime. Hibernate's `create-drop` strategy automatically generates and drops tables on application startup and shutdown[Figure 3.1]

3.3.2 Schema Structure

The database schema is designed around three primary entities:

- **Event** — `eventId` (PK), `name`, `description`, `date`, `startTime`, `endTime`, `location`, `department`, `eventType` (enum), `maxCapacity`, `organizer`
- **Registration** — `registrationId` (PK), `studentName`, `studentEmail`, `eventId` (FK), `registrationDate`, `otpVerified`
- **EventType** — Enum: `SEMINAR`, `WEBINAR`, `WORKSHOP`, `LECTURE`, `CONFERENCE`, `CULTURAL`, `HACKATHON`, and more

Relationships: *One Event* $\rightarrow$ *Many Registrations* (one-to-many with FK on Registration).

3.3.3 Tables Created

- **EVENT** — Stores all campus event details created by administrators
- **REGISTRATION** — Stores student registration records linked to events

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The application follows the **Model-View-Controller (MVC) architecture** which separates the application into three layers: the **Model** (JPA entities and data), the **View** (Thymeleaf HTML templates), and the **Controller** (Spring MVC controllers handling HTTP requests). This separation ensures maintainability, scalability, and clean code organization.[Figure 4.1]

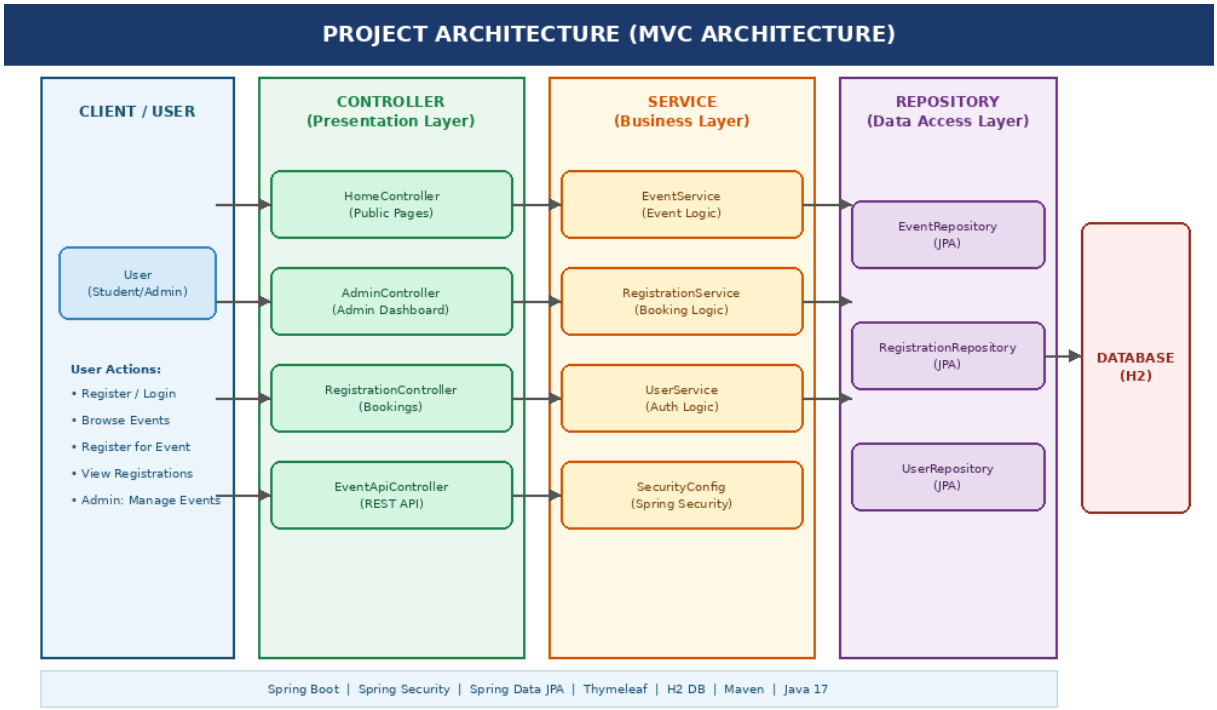

Figure 4.1: MVC Architecture of the Smart Campus Event Management System

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) at Level 0 represents how data moves through the system between users and the four primary sub-processes: User Management, Event Management, Registration, and Authentication[Figure 4.1].

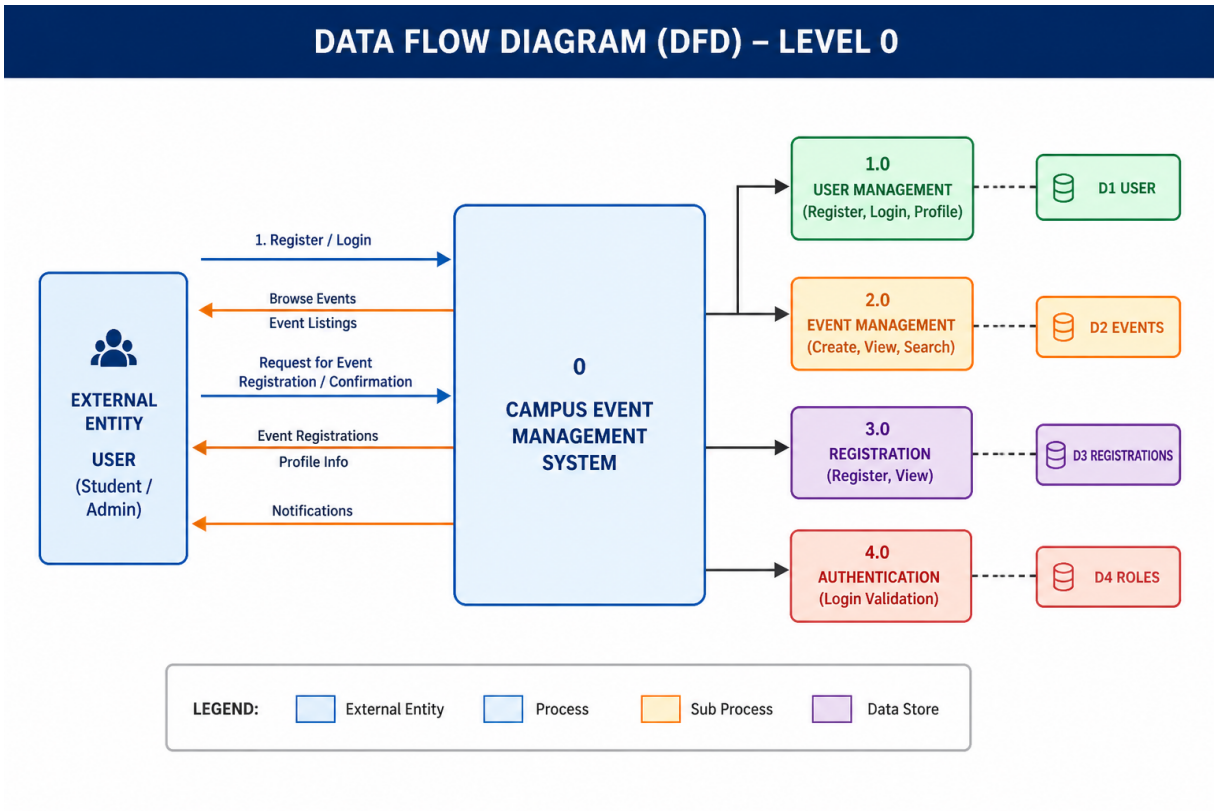

Figure 4.2: Data Flow Diagram (DFD Level 0) of the Campus Event Management System

4.3 Module 1: User Management

The User Management module handles authentication and authorization using **Spring Security**. The `SecurityConfig` class defines access rules:

- `/admin/**` endpoints require the `ADMIN` role and redirect unauthenticated users to `/admin/login`
- Public endpoints (`/`, `/events`, `/events/**`, `/api/**`) are accessible to all users
- Passwords are encoded using `BCryptPasswordEncoder`

The admin credentials (default: `admin / admin123`) are stored securely in the security configuration and verified during login. Student users register via OTP email verification without requiring persistent user accounts.

4.4 Module 2: Event Management (Core Processing)

The Event Management module is the core of the system. The `AdminController` provides secured endpoints for administrators to:

- **Create events** — `POST /admin/events/create`: Accepts event name, description, date, time, location, department, event type, and maximum capacity
- **Update events** — `POST /admin/events/edit/{id}`: Updates event details
- **Delete events** — `POST /admin/events/delete/{id}`: Removes an event
- **View dashboard** — `GET /admin/dashboard`: Analytics including total events (10), upcoming events (10), total registrations (16), and event categories (8)

The `HomeController` powers the student-facing home page, displaying trending events and platform statistics, while the `EventApiController` provides REST endpoints for live filtering and asynchronous data loading.

4.5 Module 3: Registration and Database Management

This module handles student event registration with OTP verification:

- Student clicks **"Register Now"** on an event detail page
- `RegistrationController` validates seat availability (checks `maxCapacity` vs. current registration count)
- The `OtpService` generates a 6-digit OTP and `EmailService` sends it to the student's email
- Student enters the OTP on the verification page; `RegistrationController` validates it

- On success, a `Registration` entity is saved to the H2 database with `otpVerified=true`
- A confirmation page is displayed to the student

Advantages of this Project

a) **Efficient Event Management:** Administrators manage the full event lifecycle from a centralized, analytics-powered dashboard

b) **OTP-Based Security:** Email OTP verification ensures only legitimate students complete registration

c) **Real-Time Tracking:** Live seat availability and registration counts prevent overbooking

d) **RESTful API:** `EventApiController` enables future mobile app integration

e) **Role-Based Access:** Spring Security ensures strict separation of student and admin functionality

Disadvantages

- H2 in-memory database resets on each restart; unsuitable for production without migration
- Requires an active SMTP server for OTP email delivery
- No payment gateway for paid events in the current version
- Dependent on local server; no cloud deployment in the current iteration

Chapter 5

RESULTS AND DISCUSSIONS

The Smart Campus Event Management System was successfully developed and tested across all modules. The system provides a polished, dark-themed interface for both students and administrators, enabling smooth interaction with the platform. All core functionalities were implemented and verified: student event browsing and OTP-based registration, administrator event management, and real-time dashboard analytics[Table 5.1].

The application was tested with 10 live events across 8 event categories, 16 total registrations, and verified OTP email delivery. The results confirm that the system effectively reduces manual effort and provides an accessible, secure campus event management solution.[8][10]

Table 5.1: Comparison of Existing Systems and Proposed System

Feature	**Eventbrite**	**Cvent**	**Whova**	**SCEMS (This Project)**
Role-Based Access	No	Partial	No	**Yes**
Campus-Specific	No	No	No	**Yes**
OTP Verification	No	No	No	**Yes**
REST API	Partial	Yes	No	**Yes**
Admin Dashboard	Basic	Yes	Yes	**Yes**
Event Filtering	Yes	Yes	Yes	**Yes**
Cost	Paid	Enterprise	Paid	**Free**

Output Screenshots

The following figures present the actual output screens of the Smart Campus Event Management System as captured during testing.

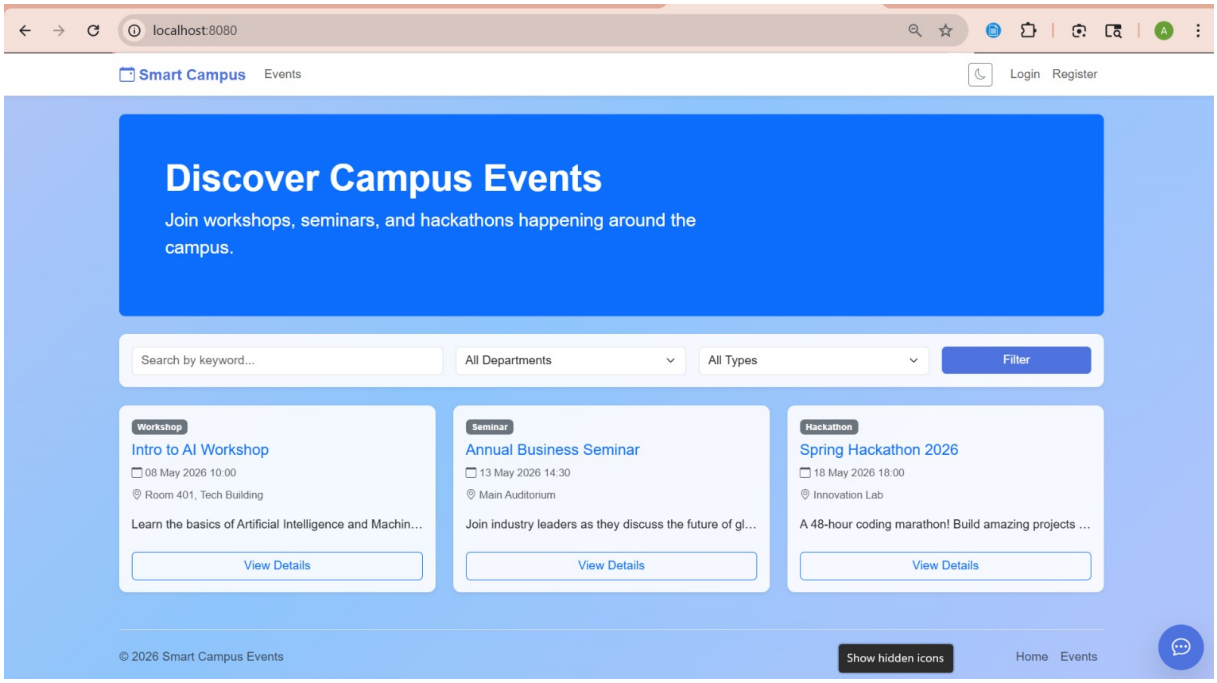

Figure 5.1: Home Page — Trending Events, Platform Statistics, and Navigation

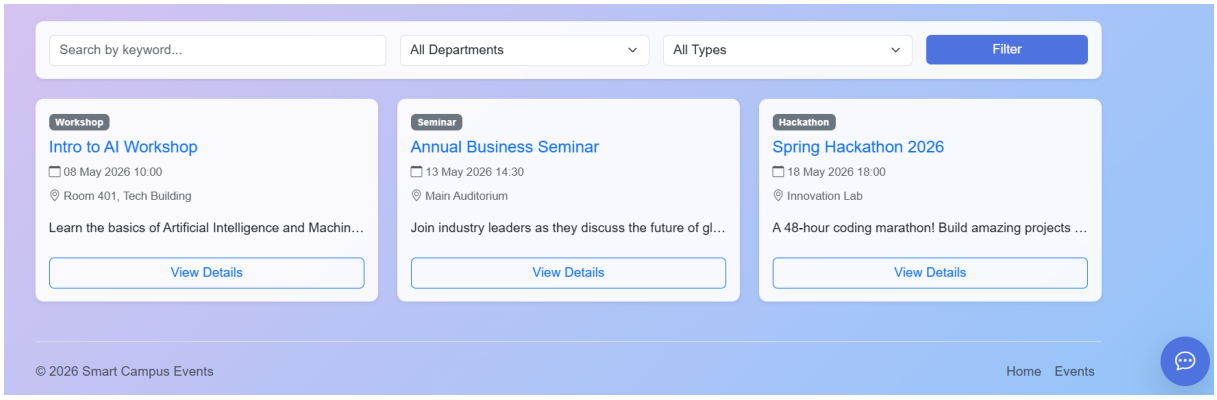

Figure 5.2: Events Listing Page — Search, Filter by Department, Type, and Date

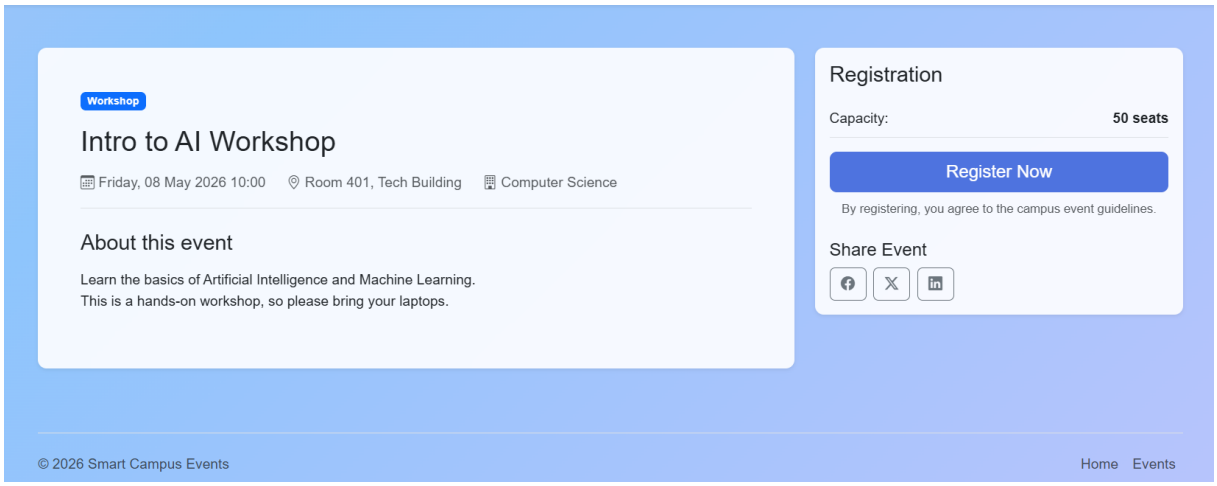

Figure 5.3: Event Detail Page — Cloud Computing & DevOps Webinar with Registration Panel

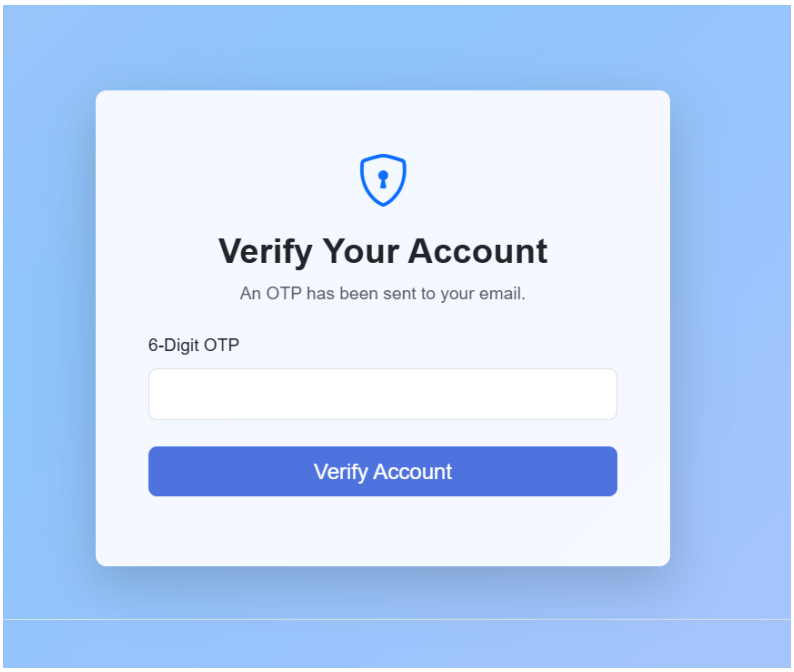

Figure 5.4: OTP Verification Page — 6-Digit Email OTP for Secure Registration

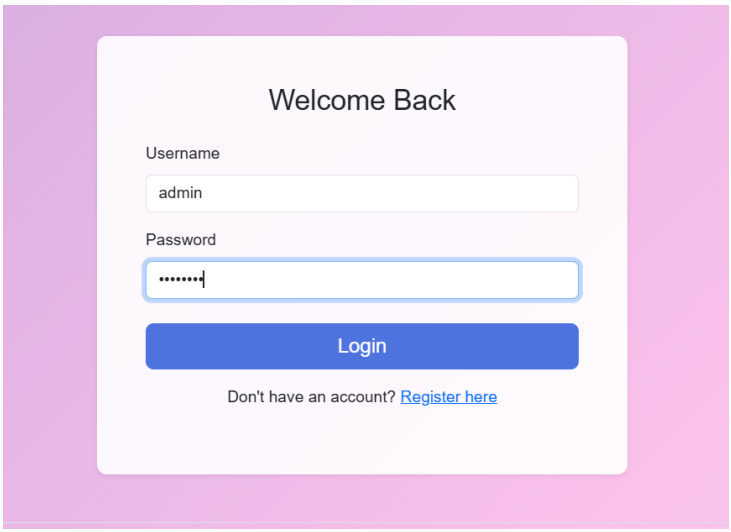

Figure 5.5: Admin Portal Login — Secure Authentication for Administrators

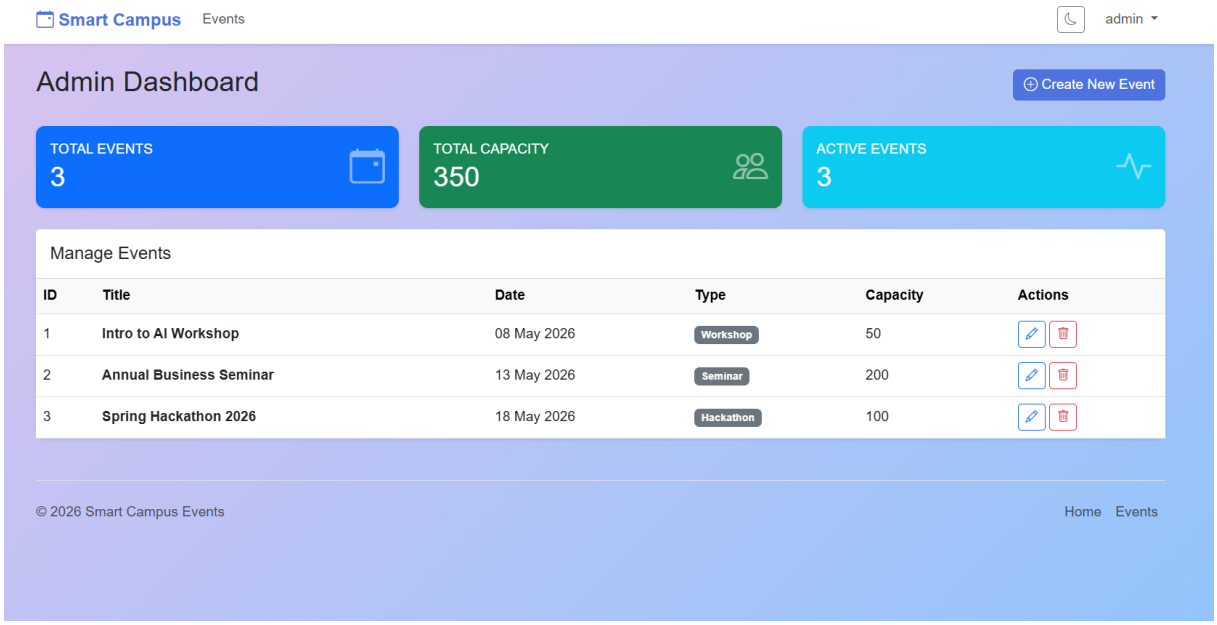

Figure 5.6: Admin Dashboard — Analytics, Events by Type, and Top Events by Registrations

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System was successfully designed, developed, and tested as a comprehensive full-stack web application. The project demonstrates the effective use of modern Java technologies — **Spring Boot 3.2.4**, **Spring Security**, **Spring Data JPA**, **Thymeleaf**, and **H2 Database** — to build a scalable, secure, and user-friendly campus event management solution[5][4].

The system fulfills all core objectives: role-based authentication for students and administrators, OTP-based email verification for secure registration,[6]real-time seat tracking and availability, administrator event management, and a comprehensive analytics dashboard. Tested with 10 events, 8 event categories, and 16 registrations, the system demonstrates reliable performance and smooth data flow across all layers.

The MVC architecture ensures clean separation of concerns, and the layered backend design maintains modularity. The RESTful API layer provides a strong foundation for future mobile application integration. Overall, the project successfully replaces manual campus event management with an efficient, transparent, and accessible digital solution[2]

6.2 Future Enhancements

The following enhancements are planned for future iterations:

- **Cloud Deployment** — Migrate from H2 to MySQL/PostgreSQL and deploy on AWS, Azure, or Heroku
- **Payment Integration** — Integrate Razorpay or Stripe for paid event ticketing
- **QR Code Tickets** — Generate QR code e-tickets for registered students for physical check-in
- **Email Notifications** — Automated event reminders and cancellation alerts using Spring Mail
- **Mobile Application** — Android/iOS app consuming the existing `EventApiController` REST endpoints
- **Advanced Analytics** — Dashboards with registration trends, attendance heatmaps, and department-wise reports
- **Virtual/Hybrid Events** — Support Zoom/Teams links and hybrid attendance tracking
- **Student Profiles** — Persistent student accounts with registration history and preferences

[12pt]report

[a4paper, margin=1in]geometry graphicx array

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of knowledge	Requires in depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge in Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies (Thymeleaf, HTML/CSS) for developing a scalable event management system.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security by ensuring easy student access while maintaining secure admin authentication and data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves system design, database schema creation, validation handling, and efficient event filtering and registration mechanisms.
WP4	Familiarity	Partially new problems	WK4, WK8	Integrating real-time event updates, user interaction, and admin analytics introduces challenges beyond traditional manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event filtering, registration tracking, and statistics generation not fully covered by standard templates.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Addresses requirements of students, faculty coordinators, and administrators ensuring smooth event organization and participation.
WP7	Interdependence	System level integration	WK3, WK5, WK6	Integrates frontend UI, backend services, database, validation, and security modules into a unified system.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management.

Table 7.3: SDG Mapping for the Project

References

[1] Spring Boot 3.2.4 Reference Documentation, Pivotal Software, 2024.
Available at: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/

[2] Spring Security Reference Documentation, Spring Framework.
Available at: https://docs.spring.io/spring-security/reference/

[3] Spring Data JPA Reference Documentation, Spring Framework.
Available at: https://docs.spring.io/spring-data/jpa/docs/current/reference/html/

[4] Thymeleaf — Using Thymeleaf (Tutorial), Thymeleaf Documentation.
Available at: https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html

[5] H2 Database Engine Documentation.
Available at: https://www.h2database.com/html/main.html

[6] Project Lombok Documentation.
Available at: https://projectlombok.org/

[7] Apache Maven Project Documentation.
Available at: https://maven.apache.org/guides/

[8] Eventbrite — Online Event Management Platform.
Available at: https://www.eventbrite.com

[9] Cvent — Enterprise Event Management Software.
Available at: https://www.cvent.com

[10] Whova — Event Management & Attendee Engagement Platform.
Available at: https://whova.com

[11] Accelevents — All-in-One Virtual & Hybrid Event Platform.
Available at: https://accelevents.com

[12] Oracle Java 17 Documentation.
Available at: https://docs.oracle.com/en/java/javase/17/

[13] W3Schools HTML5 Tutorial.
Available at: https://www.w3schools.com/html/

[14] W3Schools CSS3 Tutorial.
Available at: https://www.w3schools.com/css/

[15] GeeksforGeeks Spring Boot Guide.
Available at: https://www.geeksforgeeks.org/spring-boot/